BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

INTRODUCTION TO WEB PROGRAMMING

Unit 10

Advanced JavaScript

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Learning Objectives	-	-	
2	Basics of HTTP	-	-	4-12
3	HTTP Methods	-	-	13-20
4	Status Codes	-	-	21-26
5	Summary	-	-	27
6	Questions	-	1	28-29
7	Answers	-	-	29

1. INTRODUCTION

The Hypertext Transfer Protocol (HTTP) serves as the backbone of communication on the World Wide Web, facilitating the exchange of information between clients (such as web browsers) and servers. Understanding the fundamentals of HTTP is crucial for anyone involved in web development, as it forms the basis for how web applications and websites function. This chapter is dedicated to demystifying the core concepts of HTTP, including its structure, methods, and the status codes it uses to communicate the outcomes of client-server interactions.

To embark on this journey, we first explore the fundamental workings of HTTP, deciphering its request-response model, and dissecting the URL structure. HTTP, as a stateless protocol, relies on a series of requests and responses to transmit data over the internet. We delve into the critical elements of an HTTP message, highlighting the roles of headers and bodies. Furthermore, we examine the key components of Uniform Resource Locators (URLs) that enable clients to locate and access resources on the web. Armed with this foundational knowledge, we then move on to exploring HTTP methods and status codes, shedding light on the essential building blocks that power the web.

1.1 Learning objectives

By the end of this chapter you will be able to:

- *Describe the core components of an HTTP request and an HTTP response.*
- *Differentiate between HTTP request methods (e.g., GET, POST, PUT, DELETE) and their specific purposes in client-server interactions.*
- *Demonstrate the ability to construct HTTP requests using various methods, headers, and URL parameters.*
- *Evaluate the advantages and limitations of different HTTP methods, considering factors such as idempotence, safety, and data transfer.*
- *Design and implement HTTP-based solutions for specific web development tasks, utilizing HTTP methods and status codes to optimize performance and user experience.*

2. BASICS OF HTTP

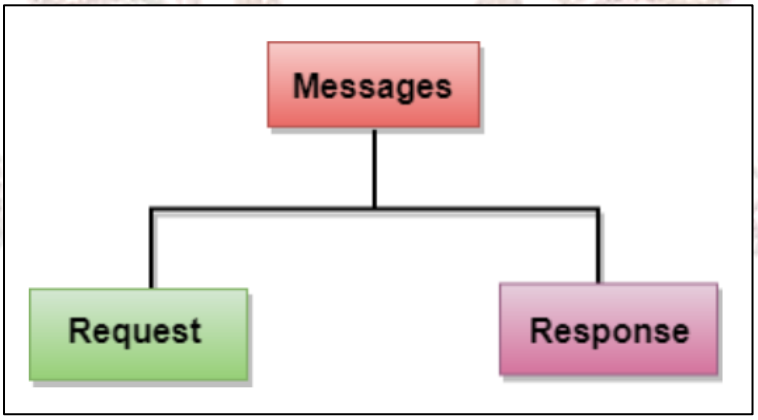
HTTP, which stands for Hypertext Transfer Protocol, is the foundation of data communication on the World Wide Web. It is an application-layer protocol that defines the rules for requesting and transmitting hypertext (text with links) documents on the web. HTTP operates in a client-server model, where web browsers (clients) request resources, and web servers respond with those resources. Here are the basics of HTTP:

1. *HTTP is a Stateless Protocol:*

HTTP (Hypertext Transfer Protocol) operates as a stateless protocol. This means that each interaction between a client (e.g., a web browser) and a server is independent of previous interactions. In other words, each HTTP request from a client to a server is treated as a separate, isolated transaction. The server does not retain any information about past requests from the same client.

This statelessness simplifies the design and implementation of web applications and servers. It also means that each request must contain all the necessary information for the server to understand and fulfill the request, including details about the resource being requested and any required parameters.

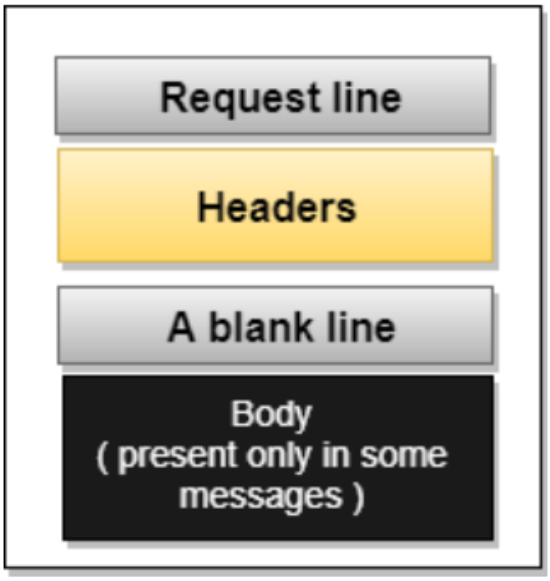
2. *HTTP Request-Response Cycle:*



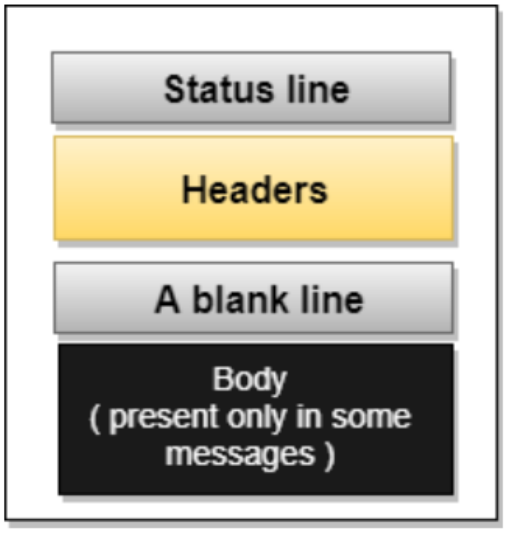
The core interaction in HTTP involves a request-response cycle:

- HTTP Request: A client initiates communication by sending an HTTP request to a server. The request includes a method (such as GET, POST, PUT, or DELETE) that

indicates the desired action, a Uniform Resource Locator (URL) specifying the resource to be accessed, and optional headers providing additional information.



- Web Server Processing: Upon receiving the HTTP request, the web server processes it based on the specified method and URL. The server may perform various tasks, such as retrieving data from a database, generating dynamic content, or fetching files.
- HTTP Response: After processing the request, the server sends back an HTTP response to the client. This response includes a status code (e.g., 200 for success, 404 for not found, 500 for server error), headers with metadata about the response, and, most importantly, the actual content or data requested by the client.



- Client Processing: The client (e.g., a web browser) receives the HTTP response. It can then parse the response to extract and display the content. For example, if the response contains HTML content, the browser renders the web page accordingly.

This request-response cycle is at the heart of web communication. It allows clients to retrieve web pages, images, videos, and other resources from remote servers, enabling the seamless browsing experience we are familiar with.

In summary, HTTP is a stateless protocol that facilitates communication between clients and servers over the World Wide Web. It operates on a request-response model, where clients send requests specifying the desired action and resources, and servers respond with the requested data. The statelessness of HTTP simplifies web development but requires that each request contains all necessary information.

Applications of HTTP:

1. *Web Browsing:*

HTTP plays a central role in web browsing, making it possible for users to access and interact with websites and web applications. Here's how it works:

- When you enter a URL (Uniform Resource Locator) into your web browser's address bar and press Enter, your browser sends an HTTP request to the web server hosting the requested website.

- The server processes the request and generates an HTTP response, which typically includes HTML, CSS, JavaScript, and other assets required to render the web page.
- Your browser receives the HTTP response, parses the content, and displays it as a fully rendered web page. This process allows you to view text, images, videos, and interact with web forms and applications.

HTTP's role in web browsing is critical because it facilitates the seamless retrieval and rendering of web content. It ensures that web pages are delivered efficiently from servers to clients, enabling users to explore and navigate the World Wide Web.

2. Web Services:

HTTP is widely used in web services, which are software systems that communicate with each other over the internet to perform various tasks. Web services leverage HTTP for data exchange, and they are crucial for enabling interoperability between different software systems and platforms. Here's how web services use HTTP:

- A client application sends an HTTP request to a server hosting a web service, specifying the desired action (e.g., retrieving data, updating records) and providing any required parameters or data.
- The web service processes the HTTP request, performs the requested operation, and generates an HTTP response, often in a structured data format like JSON or XML.
- The client application receives the HTTP response and extracts the relevant data or information from it, allowing it to integrate and utilize the web service's functionality.

Web services are commonly used in scenarios like accessing databases, retrieving weather data, or integrating with third-party platforms (e.g., payment gateways, social media APIs).

They enable different software systems, regardless of their technology stack, to communicate and share data effectively.

3. APIs (Application Programming Interfaces):

APIs are sets of rules and protocols that allow one software application to interact with and access the functionality or data of another application or service. HTTP is often used as the communication protocol in web APIs. Here's how APIs utilize HTTP:

- An API provider exposes a set of HTTP endpoints, each representing a specific function or resource. Clients can send HTTP requests to these endpoints to interact with the API.
- Clients, which can be web applications, mobile apps, or other software, send HTTP requests to the API endpoints, specifying the desired operation (e.g., retrieving user data, posting a comment).
- The API server processes the HTTP request, executes the requested operation, and returns an HTTP response, typically in a structured data format like JSON or XML, containing the requested data or confirming the action. APIs are used extensively in modern software development to enable integration and functionality sharing. They allow developers to leverage the features and data of external services and platforms, leading to more powerful and feature-rich applications.

In summary, HTTP is a versatile protocol that underpins various aspects of the internet, including web browsing, web services, and APIs. It enables the seamless exchange of data and facilitates interactions between clients and servers, making it a fundamental technology for modern web and application development.

History of HTTP:

Tim Berners Lee and his team at CERN get credit for inventing original HTTP and associated technologies.

- HTTP version 0.9: This was the first version of HTTP which was introduced in 1991.
- HTTP version 1.0: In 1996, RFC 1945 (Request For Comments) was introduced in HTTP version 1.0.
- HTTP version 1.1: In January 1997, RFC 2068 was introduced in HTTP version 1.1. Improvements and updates to the HTTP version 1.1 standard were released under RFC

2616 in June 1999.

- HTTP version 2.0: The HTTP version 2.0 specification was published as RFC 7540 on May 14, 2015.
- HTTP version 3.0: HTTP version 3.0 is based on the previous RFC draft. It is renamed as Hyper-Text Transfer Protocol QUIC which is a transport layer network protocol developed by Google.

Advantages of HTTP:

1. *Simple and Human-Readable:*

HTTP's simplicity and human-readable nature are among its primary advantages. The protocol uses plain text for both requests and responses, making it easy for developers to read, understand, and work with. Here's why this advantage is significant:

- Debugging and Troubleshooting: When developing web applications or working with HTTP-based services, developers can inspect HTTP requests and responses directly in their plain text form. This transparency simplifies the process of identifying and diagnosing issues, such as incorrect headers or unexpected content.
- Accessibility: HTTP's human-readable format allows developers to create and test requests manually using tools like web browsers, command-line utilities, or specialized software like Postman. This accessibility enhances the ability to experiment with APIs and services.
- Documentation and Learning: HTTP's simplicity facilitates the creation of clear and concise documentation for APIs and web services. It also makes it easier for newcomers to learn and understand the protocol's fundamentals.

2. *Platform-Independent:*

HTTP is platform-independent, meaning it can be used with various operating systems, programming languages, and technologies. This platform-agnostic nature has several advantages:

- Cross-Compatibility: HTTP requests and responses can be generated and interpreted by a wide range of software, regardless of the underlying infrastructure. This enables interoperability between different systems and platforms.

- Choice of Development Tools: Developers have the flexibility to choose from a variety of programming languages, libraries, and frameworks for building HTTP-based applications. This freedom allows them to select tools that align with project requirements and team expertise.
- Scalability: HTTP's platform-independence makes it well-suited for distributed and scalable systems, where components may run on diverse platforms and environments.

3. Extensible:

HTTP is extensible, meaning developers can enhance and adapt the protocol to meet specific requirements. This extensibility is realized through the use of custom headers, as well as the support for different HTTP methods. Here's how this advantage plays out:

- Custom Headers: HTTP allows developers to include custom headers in requests and responses. These headers can convey additional information or instructions specific to the application or service. For example, an API might use custom headers for authentication or data formatting.
- HTTP Methods: HTTP defines a set of standard methods like GET, POST, PUT, and DELETE. However, developers can create and use custom methods to perform specialized actions. This flexibility enables the design of RESTful APIs with clear and meaningful semantics.
- Adaptability: The ability to extend HTTP makes it adaptable to a wide range of use cases. Developers can tailor the protocol to suit the needs of specific applications or services, which is essential for achieving efficient and effective communication between clients and servers.

In summary, HTTP's advantages, including simplicity, platform-independence, and extensibility, contribute to its widespread adoption and effectiveness in powering web communication and modern applications. These characteristics enable developers to work with HTTP effectively, troubleshoot issues efficiently, and adapt the protocol to meet evolving requirements.

Disadvantages of HTTP:

1. *Statelessness:*

One of the significant disadvantages of HTTP is its statelessness. In a stateless protocol, each request-response cycle is independent, meaning the server does not retain any information about past interactions with a client. This has implications for certain applications:

- Session Management: HTTP does not inherently support session management. In applications where maintaining user sessions or maintaining state across multiple requests is essential (e.g., e-commerce carts, user logins), developers must implement mechanisms like cookies or server-side sessions. This adds complexity to application development.
- Inefficient Context Handling: Stateless nature makes it challenging to maintain context between requests. For example, when a user navigates through multiple pages on a website, the server doesn't remember their previous interactions. Developers need to design workarounds to provide a seamless user experience.

2. *Security:*

HTTP does not provide data encryption by default, making it vulnerable to eavesdropping and interception by malicious actors. Here's a closer look at the security concerns:

- Data Exposure: Data transmitted over plain HTTP is exposed in plaintext form, which means that sensitive information like login credentials, personal data, or financial details can be intercepted and read by unauthorized parties.
- Man-in-the-Middle Attacks: Attackers can exploit the lack of encryption to perform man-in-the-middle (MITM) attacks, intercepting and altering the data exchanged between the client and server.
- Data Integrity: Without encryption, there is no guarantee that data received by the client is unchanged and has not been tampered with during transit.

To address these security concerns, HTTPS (HTTP Secure) was introduced. HTTPS encrypts data using protocols like TLS (Transport Layer Security) to ensure secure communication between clients and servers. HTTPS has become the standard for secure web communication.

3. Performance:

HTTP's stateless nature and associated overhead can impact the performance of certain applications, particularly those requiring high-performance real-time data transfer:

- Connection Overhead: For each HTTP request, there is the overhead of establishing a new connection with the server. This can be significant in scenarios where numerous small requests are made, as it incurs additional latency.
- Lack of Persistent Connections: HTTP traditionally uses short-lived connections, where the connection is closed after each request-response cycle. While HTTP/1.1 introduced persistent connections to address this, it may not be as efficient as other protocols, such as WebSockets, for real-time applications.
- High Header Overhead: HTTP headers can add overhead to each request and response, especially when the payload is relatively small. This can impact bandwidth utilization and response times.

In cases where performance is critical, alternative protocols like WebSockets, which provide full-duplex communication with lower overhead, may be preferred for real-time data transfer.

In summary, HTTP's disadvantages, including statelessness, security vulnerabilities in plain HTTP, and potential performance limitations, have led to the development and adoption of technologies like HTTPS and alternative protocols to address these shortcomings and enhance the security and efficiency of web communication.

3. HTTP METHODS

The set of common methods for HTTP/1.1 is defined below and this set can be expanded based on requirements. These method names are case sensitive and they must be used in uppercase.

HTTP methods (also known as HTTP request methods) are a fundamental part of the HTTP protocol, defining the type of action a client (usually a web browser or application) wants to perform on a resource located on a web server. Each HTTP request typically includes one of these methods to specify the desired action. Here, we'll explain the most commonly used HTTP methods in detail:

1. *GET:*
   - Purpose: The GET method is used to request data from a specified resource. It retrieves data without causing any side effects on the server or modifying the resource.
   - Usage: GET is commonly used for fetching web pages, images, style sheets, and other types of resources from web servers. It appends data to the URL in the form of query parameters.

A GET request retrieves data from a web server by specifying parameters in the URL portion of the request. This is the main method used for document retrieval. The following example makes use of GET method to fetch hello.htm:

GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.example.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

The server response against the above GET request will be as follows:

HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)

Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT

ETag: "34aa387-d-1568eb00"

Vary: Authorization,Accept

Accept-Ranges: bytes

Content-Length: 88

Content-Type: text/html

Connection: Closed

<html>

<body>

<h1>Hello, World!</h1>

</body>

</html>

2. *POST:*
   - Purpose: The POST method is used to submit data to be processed to a specified resource. It can create new resources, update existing ones, or perform other server-side actions.
   - Usage: POST is often used for submitting forms on web pages, uploading files, and making complex requests that involve sending data to the server in the request body.

The POST method is used when you want to send some data to the server, for example, file update, form data, etc. The following example makes use of POST method to send a form data to the server, which will be processed by a process.cgi and finally a response will be returned:

POST /cgi-bin/process.cgi HTTP/1.1

User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)

Host: www.example.com

Content-Type: text/xml; charset=utf-8

Content-Length: 88

Accept-Language: en-us

Accept-Encoding: gzip, deflate

Connection: Keep-Alive

<?xml version="1.0" encoding="utf-8"?>

<string xmlns="http://clearforest.com/">string</string>

The server-side script process.cgi processes the passed data and sends the following response:

HTTP/1.1 200 OK

Date: Mon, 27 Jul 2009 12:28:53 GMT

Server: Apache/2.2.14 (Win32)

Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT

ETag: "34aa387-d-1568eb00"

Vary: Authorization,Accept

Accept-Ranges: bytes

Content-Length: 88

Content-Type: text/html

Connection: Closed

<html>

<body>

<h1>Request Processed Successfully</h1>

</body>

</html>

3. *PUT:*
   - Purpose: The PUT method is used to update a specified resource or create a new resource if it does not exist at the specified URL.
   - Usage: PUT is commonly used for updating data, such as modifying the content of a web page or replacing an existing resource with a new one.

The PUT method is used to request the server to store the included entity-body at a location specified by the given URL. The following example requests the server to save the given entity-body in hello.htm at the root of the server:

PUT /hello.htm HTTP/1.1

User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)

Host: www.example.com

Accept-Language: en-us

Connection: Keep-Alive

Content-type: text/html

Content-Length: 182

<html>

<body>

<h1>Hello, World!</h1>

</body>

</html>

The server will store the given entity-body in hello.htm file and will send the following response back to the client:

HTTP/1.1 201 Created

Date: Mon, 27 Jul 2009 12:28:53 GMT

Server: Apache/2.2.14 (Win32)

Content-type: text/html

Content-length: 30

Connection: Closed

<html>

<body>

<h1>The file was created.</h1>

</body>

</html>

4. *DELETE:*
   - Purpose: The DELETE method is used to request the removal of a specified resource.
   - Usage: DELETE is often used when a client wants to delete a resource, such as deleting a file on a server or removing a record from a database.

The DELETE method is used to request the server to delete a file at a location specified by the given URL. The following example requests the server to delete the given file hello.htm at the root of the server:

DELETE /hello.htm HTTP/1.1

User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)

Host: www.example.com

Accept-Language: en-us

Connection: Keep-Alive

The server will delete the mentioned file hello.htm and will send the following response back to the client:

HTTP/1.1 200 OK

Date: Mon, 27 Jul 2009 12:28:53 GMT

Server: Apache/2.2.14 (Win32)

Content-type: text/html

Content-length: 30

Connection: Closed

<html>

<body>

<h1>URL deleted.</h1>

</body>

</html>

5. *PATCH:*
   - Purpose: The PATCH method is used to apply partial modifications to a resource. It is typically used to update specific attributes of a resource without replacing it entirely.
   - Usage: PATCH is useful when making minor updates to a resource, such as editing a single field in a database record.
6. *HEAD:*
   - Purpose: The HEAD method is similar to GET but is used to retrieve metadata about a resource without actually fetching the resource's content. It is often used to check resource availability or obtain information like the content type and size.
   - Usage: HEAD requests are useful for reducing bandwidth usage when only metadata is required.

The HEAD method is functionally similar to GET, except that the server replies with a response line and headers, but no entity-body. The following example makes use of HEAD method to fetch header information about hello.htm:

HEAD /hello.htm HTTP/1.1

User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)

Host: www.example.com

Accept-Language: en-us

Accept-Encoding: gzip, deflate

Connection: Keep-Alive

The server response against the above HEAD request will be as follows:

HTTP/1.1 200 OK

Date: Mon, 27 Jul 2009 12:28:53 GMT

Server: Apache/2.2.14 (Win32)

Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT

ETag: "34aa387-d-1568eb00"

Vary: Authorization,Accept

Accept-Ranges: bytes

Content-Length: 88

Content-Type: text/html

Connection: Closed

You can notice that here server the does not send any data after header.

7. *OPTIONS:*
   - Purpose: The OPTIONS method is used to retrieve information about the communication options available for a resource. It can be used to determine the supported HTTP methods, headers, and other communication details.
   - Usage: OPTIONS requests help clients discover the capabilities of a server or resource, which is particularly useful for building more robust and flexible APIs.

The OPTIONS method is used by the client to find out the HTTP methods and other options supported by a web server. The client can specify a URL for the OPTIONS method, or an asterisk () to refer to the entire server. The following example requests a list of methods supported by a web server running on example.com:

OPTIONS HTTP/1.1

User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)

The server will send an information based on the current configuration of the server, for example:

HTTP/1.1 200 OK

Date: Mon, 27 Jul 2009 12:28:53 GMT

Server: Apache/2.2.14 (Win32)

Allow: GET,HEAD,POST,OPTIONS,TRACE

Content-Type: httpd/unix-directory

8. *CONNECT, TRACE, and Others:*

These methods are less commonly used in regular web development:

- CONNECT: Primarily used for setting up network connections through proxy servers.
- TRACE: Used to retrieve a diagnostic trace of a request's path through intermediary servers, mainly for debugging purposes.

a. *CONNECT Method*

The CONNECT method is used by the client to establish a network connection to a web server over HTTP. The following example requests a connection with a web server running on the host example.com:

CONNECT www.example.com HTTP/1.1

User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)

The connection is established with the server and the following response is sent back to the client:

HTTP/1.1 200 Connection established

Date: Mon, 27 Jul 2009 12:28:53 GMT

Server: Apache/2.2.14 (Win32)

b. *TRACE Method*

The TRACE method is used to echo the contents of an HTTP Request back to the requester which can be used for debugging purpose at the time of development. The following example shows the usage of TRACE method:

TRACE / HTTP/1.1

Host: www.example.com

User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)

The server will send the following message in response to the above request:

HTTP/1.1 200 OK

Date: Mon, 27 Jul 2009 12:28:53 GMT

Server: Apache/2.2.14 (Win32)

Connection: close

Content-Type: message/http

Content-Length: 39

TRACE / HTTP/1.1

Host: www.example.com

User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)

HTTP methods are essential for specifying the intent of a client's request and determining the appropriate action to be taken on the server. Web developers use these methods to create dynamic and interactive web applications, RESTful APIs, and other web services, depending on the specific use case and requirements of their applications.

4. HTTP STATUS CODES

HTTP status codes are three-digit numeric codes returned by web servers to indicate the outcome of an HTTP request made by a client, such as a web browser or an application. These status codes convey information about the success, failure, or other conditions of the request-response cycle. They help both clients and servers understand and respond to requests effectively. HTTP status codes are grouped into several classes, each representing a different category of response.

The Status-Code element in a server response, is a 3-digit integer where the first digit of the Status-Code defines the class of response and the last two digits do not have any categorization role. There are 5 values for the first digit:

S.N.	Code and Description
1	1xx: Informational It means the request has been received and the process is continuing.
2	2xx: Success It means the action was successfully received, understood, and accepted.
3	3xx: Redirection It means further action must be taken in order to complete the request.
4	4xx: Client Error It means the request contains incorrect syntax or cannot be fulfilled.

5	5xx: Server Error It means the server failed to fulfill an apparently valid request.

HTTP status codes are extensible and HTTP applications are not required to understand the meaning of all the registered status codes. Given below is a list of all the status codes.

1xx: Information

Message	Description
100 Continue	Only a part of the request has been received by the server, but as long as it has not been rejected, the client should continue with the request.
101 Switching Protocols	The server switches protocol.

2xx: Successful

Message	Description
200 OK	The request is OK.
201 Created	The request is complete, and a new resource is created .
202 Accepted	The request is accepted for processing, but the processing is not complete.
203 Non-authoritative Information	The information in the entity header is from a local or third-party copy, not from the original server.

204 No Content	A status code and a header are given in the response, but there is no entity-body in the reply.
205 Reset Content	The browser should clear the form used for this transaction for additional input.
206 Partial Content	The server is returning partial data of the size requested. Used in response to a request specifying a *Range* header. The server must specify the range included in the response with the *Content-Range* header.

3xx: Redirection

Message	Description
300 Multiple Choices	A link list. The user can select a link and go to that location. Maximum five addresses .
301 Moved Permanently	The requested page has moved to a new url .
302 Found	The requested page has moved temporarily to a new url .
303 See Other	The requested page can be found under a different url .
304 Not Modified	This is the response code to an *If-Modified-Since* or *If-None-Match* header, where the URL has not been modified since the specified date.

305 Use Proxy	The requested URL must be accessed through the proxy mentioned in the *Location* header.
306 *Unused*	This code was used in a previous version. It is no longer used, but the code is reserved.
307 Temporary Redirect	The requested page has moved temporarily to a new url.

4xx: Client Error

Message	Description
400 Bad Request	The server did not understand the request.
401 Unauthorized	The requested page needs a username and a password.
402 Payment Required	*You can not use this code yet.*
403 Forbidden	Access is forbidden to the requested page.
404 Not Found	The server can not find the requested page.
405 Method Not Allowed	The method specified in the request is not allowed.
406 Not Acceptable	The server can only generate a response that is not accepted by the client.

407 Proxy Authentication Required	You must authenticate with a proxy server before this request can be served.
408 Request Timeout	The request took longer than the server was prepared to wait.
409 Conflict	The request could not be completed because of a conflict.
410 Gone	The requested page is no longer available .
411 Length Required	The "Content-Length" is not defined. The server will not accept the request without it .
412 Precondition Failed	The pre condition given in the request evaluated to false by the server.
413 Request Entity Too Large	The server will not accept the request, because the request entity is too large.
414 Request-url Too Long	The server will not accept the request, because the url is too long. Occurs when you convert a "post" request to a "get" request with a long query information .
415 Unsupported Media Type	The server will not accept the request, because the mediatype is not supported .
416 Requested Range Not Satisfiable	The requested byte range is not available and is out of bounds.

417 Expectation Failed	The expectation given in an Expect request-header field could not be met by this server.

5xx: Server Error

Message	Description
500 Internal Server Error	The request was not completed. The server met an unexpected condition.
501 Not Implemented	The request was not completed. The server did not support the functionality required.
502 Bad Gateway	The request was not completed. The server received an invalid response from the upstream server.
503 Service Unavailable	The request was not completed. The server is temporarily overloading or down.
504 Gateway Timeout	The gateway has timed out.
505 HTTP Version Not Supported	The server does not support the "http protocol" version.

5. SUMMARY

This chapter provides a comprehensive overview of the core concepts of HTTP (Hypertext Transfer Protocol), covering HTTP's foundational principles, HTTP methods (request types), and HTTP status codes (response indicators). Understanding these elements is fundamental for anyone involved in web development, as they form the basis for how web communication functions. The chapter begins by explaining the Basics of HTTP. It introduces HTTP as a stateless protocol that facilitates communication between clients (usually web browsers) and web servers. HTTP operates on a request-response model, where clients send HTTP requests to servers to fetch or manipulate resources. The chapter elucidates the key components of an HTTP request, including the URL (Uniform Resource Locator), headers, and methods. It also outlines HTTP's stateless nature, which means each request-response cycle is independent. Chapter delves into HTTP Methods, which define the type of action a client intends to perform on a resource. It provides detailed explanations of commonly used HTTP methods, such as GET (for retrieving data), POST (for submitting data), PUT (for updating resources), DELETE (for removing resources), PATCH (for making partial updates), and more. The chapter illustrates how each method corresponds to specific use cases in web development, allowing developers to interact with web resources effectively. Finally, the chapter explores HTTP Status Codes, which are numeric codes returned by web servers to convey the outcome of an HTTP request. It categorizes status codes into classes, including Informational, Success, Redirection, Client Errors, and Server Errors, and elucidates their meanings. HTTP status codes are crucial for clients and servers to understand and respond to requests correctly, aiding in error handling, troubleshooting, and communication between web applications and users. In conclusion, this chapter equips readers with a foundational understanding of HTTP, HTTP methods, and status codes, essential for building and maintaining web applications and services. These concepts are the building blocks of modern web communication, and a firm grasp of them is indispensable for web developers, administrators, and anyone engaged in web-related endeavours.

6. QUESTIONS

Self-Assessment Questions

SELF-ASSESSMENT QUESTIONS – 1

1. What does HTTP stand for?
2. Define the term "HTTP method."
3. Which HTTP method is commonly used to retrieve data from a web server?
4. Which HTTP method is typically used for submitting data to a server?
5. What does the status code "404" indicate in HTTP?
6. In HTTP, what is the purpose of the "HEAD" method?
7. Which class of HTTP status codes represents client errors?
8. Which HTTP status code class indicates successful responses?
9. What is the primary role of the "OPTIONS" HTTP method?
10. Explain the concept of HTTP statelessness.

Terminal questions

1. Explain the key components of an HTTP request. How do headers and methods contribute to the request process?
2. Discuss the importance of HTTP methods in web development. Provide real-world examples of when to use "GET," "POST," "PUT," and "DELETE" methods in web applications.
3. Describe the significance of HTTP status codes in web communication. Provide detailed explanations for at least three common HTTP status codes, including their meanings and typical use cases.
4. In the context of HTTP, what is meant by "statelessness"? How does the statelessness of HTTP impact the design and development of web applications?
5. Examine the security implications of using plain HTTP (without encryption) for web communication. What are the risks associated with transmitting sensitive data over plain HTTP, and how can these risks be mitigated?

6. Compare and contrast the "GET" and "POST" HTTP methods. Discuss their differences in terms of data handling, security considerations, and suitability for various web application scenarios.

7. ANSWERS

Self-Assessment Questions

1. HTTP stands for "Hypertext Transfer Protocol."
2. An "HTTP method" defines the type of action a client wants to perform on a resource.
3. The "GET" HTTP method is commonly used to retrieve data from a web server.
4. The "POST" HTTP method is typically used for submitting data to a server.
5. The status code "404" in HTTP indicates that the requested resource was not found on the server.
6. The "HEAD" method in HTTP is used to retrieve metadata about a resource without fetching its content.
7. The class of HTTP status codes that represents client errors is the "4xx" class.
8. The "2xx" class of HTTP status codes indicates successful responses.
9. The "OPTIONS" HTTP method is primarily used to retrieve information about the communication options available for a resource.
10. In HTTP, "statelessness" means that each request-response cycle is independent, and the server does not retain information about past requests.

Terminal Questions

1. Refer Section 2
2. Refer Section 3
3. Refer Section 4
4. Refer Section 2
5. Refer Section 3
6. Refer Section 4